

Session 17 : TP — Véhicules (Raycast Vehicle)

Objectifs du TP

- Comprendre le modèle **Raycast Vehicle** — et pourquoi on ne simule **jamais** les roues comme des rigid bodies.
- **Phase A** : implémenter la suspension à la **main** — raycast + ressort + amortisseur, roue par roue.
- Piloter le véhicule : **accélération, freinage, direction** — au clavier.
- **Phase B** : remplacer la suspension maison par le **vehicle controller** de Rapier (DynamicRayCastVehicleController) — et comparer.
- Comprendre la **tenue de route** : grip, slip angle, et pourquoi une voiture dérape.
- **Tout réutiliser** : rigid bodies (S14), raycasts (S15), la philosophie du character controller (S16), les ressorts (S7).

💡 Le TP final

C'est la **convergence** du cours. Un véhicule, c'est un **character controller à roues** : le code décide (throttle, direction), les raycasts vérifient (où est le sol sous chaque roue ?). Entre les deux : une **suspension** — le ressort de la Session 7, piloté par le raycast de la Session 15, appliqué quatre fois par frame. Vous avez toutes les briques ; aujourd'hui, on les assemble.

Partie 1 : Le modèle Raycast Vehicle

Pourquoi pas 4 roues rigid bodies ?

L'intuition du débutant : « une roue = un rigid body cylindrique + un joint revolute ». Ça **marche** — et c'est un cauchemar :

- 4 joints à résoudre à **chaque frame** — le solveur lutte entre roue et châssis (oscillations).
- La moindre bosse **décolle** une roue — le joint s'étire, la voiture part en vrille.
- Le moindre virage **retourne** le châssis — il n'y a rien pour le tenir.

Le modèle des jeux **arcade** : les roues ne sont **pas des corps**. Ce sont des **raycasts** — quatre lignes invisibles qui interrogent le sol.

Raycast Vehicle

- Un châssis : rigid body dynamique (boîte).

Quatre roues « virtuelles » : un point d'attache sur le châssis + un **raycast** vers le bas.

- Si le rayon touche le sol : la roue **porte** — on calcule la force de suspension et on l'applique au châssis.
- Si le rayon ne touche rien : la roue est **en l'air** — pas de force, la roue **pend**.

Avantages : stable, performant, et les paramètres (raideur, amortissement, grip) se règlent **directement** — sans se battre contre le solveur.

💡 Roues visuelles, roues physiques

Les roues que le joueur **voit** (meshes cylindriques) sont **découplées** des raycasts : chaque frame, on positionne le mesh de la roue au point d'impact du rayon (ou à la position « pendante » si en l'air), et on le fait tourner selon la vitesse. Le visuel **suit** la physique — il ne la **subit** pas.

Partie 2 : Phase A — la suspension à la main

La boucle par roue

Pour chaque roue, à chaque frame physique :

1. **Raycast** depuis le point d'attache, vers le bas, de longueur $L_{\text{repos}} + r_{\text{roue}}$.
2. Si on touche le sol à distance d : la **compression** vaut $c = 1 - \frac{d}{L_{\text{repos}}}$ (0 = détendue, 1 = écrasée).
3. **Force de ressort** : $F_k = k \cdot c \cdot L_{\text{repos}}$ — le ressort de la S7.
4. **Amortisseur** : $F_c = c_{\text{damp}} \cdot v_{\text{suspension}}$ — la vitesse **verticale** du point d'attache.
5. On applique $F = F_k - F_c$ **au point d'attache** : `addForceAtPoint`.

```
// Phase A — le cœur du TP (à implémenter)
for (const wheel of wheels) {
  const ray = new RAPIER.Ray(wheel.attachWorld(), { x: 0, y: -1, z: 0 });
  const hit = world.castRay(ray, wheel.restLength + wheel.radius, true);

  if (hit) {
    const compression = 1 - hit.timeOfImpact / wheel.restLength;
    const springF = wheel.k * compression * wheel.restLength;
    const damperF = wheel.damping * wheel.attachVerticalVelocity();
    chassisBody.addForceAtPoint(
      { x: 0, y: springF - damperF, z: 0 },
      wheel.attachWorld(), true
    );
    wheel.placeAt(ray.pointAt(hit.timeOfImpact)); // visuel
  } else {
    wheel.placeDropped(); // la roue pend
  }
}
```

💡 Valeurs de départ

Châssis : cuboïde (0.9, 0.25, 1.2) (demi-tailles), densité 150 (≈ 324 kg).

Par roue : $k = 12,000$ N/m, $c_{\text{damp}} = 800$ N dot s/m, $L_{\text{repos}} = 0.5$ m, $r_{\text{roue}} = 0.35$ m.

Méthode de réglage : commencez sans amortisseur — la voiture **oscille** (c'est le symptôme).

Ajoutez c_{damp} jusqu'à ce qu'elle se stabilise en ≈ 1 rebond. Trop d'amortisseur = la voiture « plonge » et reste écrasée.

⚠ Le piège : appliquer la force au centre

`addForce` applique au **centre de masse** — la voiture ne peut plus **picher** ni **rouler**, elle glisse comme sur des rails. Il faut `addForceAtPoint` au point d'attaque de **chaque roue** : c'est ce qui fait pencher la voiture en virage et piquer au freinage. C'est le **moment** de la force (S3) en action.

Partie 3 : Piloter

Accélération et direction

```
// Chaque frame — input au clavier (WASD / flèches)
const forward = new THREE.Vector3(0, 0, -1).applyQuaternion(chassisQuat);

// Accélération : force longitudinale sur les roues MOTRICES (arrière)
```

```

if (keyW) chassisBody.addForce(forward.clone().multiplyScalar(4000), true);
if (keyS) chassisBody.addForce(forward.clone().multiplyScalar(-3000), true); //
frein/recul

// Direction : on *tourne les raycasts avant* + un couple léger pour assister
if (keyA) frontWheels.forEach(w => w.steer = 0.4); // rad
if (keyD) frontWheels.forEach(w => w.steer = -0.4);

```

La **direction** se fait en **inclinant les raycasts avant** : la force de suspension, appliquée dans l'axe incliné de la roue, pousse le châssis **de côté** — c'est ce qui fait tourner. On ajoute un couple d'assistance ($\vec{\tau}_y$) pour la réponse.

Grip et slip angle

- **Force longitudinale** : accélération / freinage — le long du plan de la roue.
- **Force latérale** : le **grip** — perpendiculaire au plan de la roue. Sans elle, la voiture glisse comme sur la glace.

Slip angle : l'angle entre la direction **du pneu** et la direction **réelle** du mouvement.

- Petit angle : la voiture tourne **proprement**.
- Grand angle : la voiture **dérape** (drift) — le grip « lâche ».

Modèle arcade simple : à chaque roue en contact, appliquer une force latérale proportionnelle à la vitesse latérale **locale** de la roue, plafonnée par μ (le grip max). Un μ élevé = karting, un μ faible = rallye sur neige.

Partie 4 : Phase B — le vehicle controller de Rapier

💡 Pourquoi changer d'outil ?

Vous venez de **sentir** chaque paramètre de la suspension — combien d'itérations pour stabiliser c_{damp} , comment un k trop mou fait toquer le châssis. C'était le but de la Phase A. En production, personne n'écrit cette boucle : le moteur la fournit. Rapier expose le **DynamicRayCastVehicleController** — le modèle Bullet (btRaycastVehicle), intégré.

Le contrôleur, en une douzaine de lignes

```

// Phase B — remplacer toute la Phase A par ceci
const controller = world.createVehicleController(chassisBody);

// 4 roues : point d'attache, direction, axe, longueur repos, rayon
const wheelDefs = [
  { pos: { x: -0.8, y: 0.0, z: -1.2 }, front: true },
  { pos: { x: 0.8, y: 0.0, z: -1.2 }, front: true },
  { pos: { x: -0.8, y: 0.0, z: 1.2 }, front: false },
  { pos: { x: 0.8, y: 0.0, z: 1.2 }, front: false },
];
for (const w of wheelDefs) {
  const index = controller.addWheel(
    w.pos, { x: 0, y: -1, z: 0 }, // direction du rayon
    { x: -1, y: 0, z: 0 }, // axe de la roue
    0.5, 0.35 // repos, rayon
  );
}

```

```

controller.setWheelSuspensionStiffness(index, 12 * 50);
controller.setWheelSuspensionRelaxation(index, 4.5);
controller.setWheelSuspensionCompression(index, 4.4);
controller.setWheelFrictionSlip(index, 10);
}

// Chaque frame – pilotage par roue
for (let i = 0; i < 4; i++) {
    controller.setWheelSteering(i, isFront[i] ? steerInput : 0);
    controller.setWheelEngineForce(i, isRear[i] ? throttle * 4000 : 0);
}
controller.updateVehicle(dt); // avant world.step()

Lecture par roue : wheelIsInContact(i), wheelSuspensionLength(i), wheelRotation(i),
currentVehicleSpeed — pour le visuel et le HUD.

```

La table de correspondance.

Votre Phase A	Contrôleur Rapier
$k \cdot c \cdot L_{\text{repos}}$	setWheelSuspensionStiffness
$c_{\text{damp}} \cdot v_{\text{susp}}$	setWheelSuspensionRelaxation / Compression
plafond de course	setWheelMaxSuspensionTravel
μ latéral	setWheelFrictionSlip
force moteur maison	setWheelEngineForce
inclinaison des raycasts	setWheelSteering

Mêmes concepts, autres noms — c'est le but : vous savez déjà ce que **chaque paramètre fait**, parce que vous venez de l'écrire.

Partie 5 : Missions

Phase A — la main sur la suspension

1. **M1 — Châssis** : boîte dynamique + sol + 4 **raycasts visibles** (lignes) + meshes de roues. La voiture tombe, les rayons s'affichent. *Check* : on voit les 4 rayons toucher le sol.
2. **M2 — Suspension** : ressort + amortisseur par roue (addForceAtPoint). *Check* : la voiture se pose et se stabilise — elle ne oscille plus, elle ne traverse pas.
3. **M3 — Pilotage** : WASD/flèches — accélération, freinage, direction. *Check* : on roule, on tourne, on ne se retourne pas à 20 km/h.
4. **M4 — Terrain** : une rampe et quelques bosses fixes. *Check* : on saute la rampe et on atterrit sans exploser.

Comptez 60-90 min de réglage — c'est normal, c'est le TP.

Phase B — l'outil du moteur

1. **M5 — Le contrôleur** : même véhicule, suspension remplacée par DynamicRayCastVehicleController. *Check* : comportement au moins aussi bon, en 15 lignes.
2. **M6 — Comparaison** : un essai par ligne — stabilité, code, réglage, feeling. *Check* : vous savez dire **pourquoi** le contrôleur est meilleur (ou pas).

Objectif : reconnaître chaque paramètre — si vous ne savez pas ce que fait `suspensionRelaxation`, relisez votre Phase A.

Défi — le circuit

Un petit circuit : rampe, virage serré, chicane. **Contrainte** : ne pas se retourner, ne pas dérailler. **Chrono** affiché au HUD. Le vainqueur du `frictionSlip` le plus bas sans sortir de piste gagne le respect éternel de la classe.

💡 Le fichier solution

`examples/session17_vehicle_solution.js` — comme toujours : **essayez d'abord**. La Phase A est courte à écrire (≈ 40 lignes) mais longue à régler — c'est en réglant qu'on apprend.